

Mercado Libre DataSec Technical Challenge

Instructions

- Create a separate file for each question with the exact filename specified in each challenge.
 - Commit all the solutions to a single public GitHub repository.
 - Language Requirements:
 - Challenges 1 and 2: Use Python 3.12 (specify this version in your code comments)
 - Challenge 3: Use MySQL 8.x syntax (pure SQL query file)
 - Challenge 4: Use the language specified by your evaluator (see separate Challenge 4 document)
 - IMPORTANT (Challenges 1–3): You must implement the exact function signatures, method names, parameters, and return types as specified in each challenge. Your code will be automatically tested using these exact signatures.
 - Each solution file for Challenges 1–3 must contain the exact function with the exact name, parameters, and return type as shown in the challenge description.
 - Note on additional features (Challenges 1–3): While you are welcome to add value-added features to demonstrate your skills and stand out, these enhancements must not interfere with the basic functionality, method signatures, or expected return values specified for each challenge. The core function of each solution must work exactly as specified, regardless of any additional features you implement.
 - Challenge 4 is open-ended: it does not prescribe function signatures or file structure beyond the required deliverables listed in its separate document. Your design choices are part of what is evaluated.
-

1. Minesweeper Number of Neighbouring Mines

Your solution for this challenge must be in a file named `solution_minesweeper.py`.

Create a function that takes a list representation of a Minesweeper board, and returns another board where the value of each cell is the amount of its neighbouring mines.

Requirements

Input: - A 2D list (matrix) representing a Minesweeper board - Each cell must be either 0 (empty space) or 1 (mine)

Output: - A 2D list (matrix) of the same dimensions as the input - Each cell contains the count of neighbouring mines (0-8), or 9 if the cell contains a mine

Required Function Signature

You must implement the following function exactly as specified:

```
def count_neighbouring_mines(board: list) -> list:
    """
    Counts neighbouring mines for each cell in a Minesweeper board.

    Parameters:
        board (list): A 2D list where 0 represents an empty space and 1 represents a mine

    Returns:
        list: A 2D list where each cell contains the count of neighbouring mines,
              or 9 if the cell contains a mine
    """
    # Your implementation here
```

Example

Input:

```
[
  [0, 1, 0, 0],
  [0, 0, 1, 0],
  [0, 1, 0, 1],
  [1, 1, 0, 0]
]
```

Expected Output:

```
[
  [1, 9, 2, 1],
  [2, 3, 9, 2],
  [3, 9, 4, 9],
  [9, 9, 3, 1]
]
```

Notes

- Since in the output the numbers 0-8 are used to determine the amount of adjacent mines, the number 9 will be used to identify the mines instead.
 - You can use the [Wikipedia page](#) explaining how Minesweeper works.
-

2. REST API: Best TV Shows in Genre

Your solution for this challenge must be in a file named `solution_best_in_genre.py`.

Use the HTTP GET method to retrieve information about recent television shows. Query the API:

```
https://jsonmock.hackerrank.com/api/tvseries
```

The query result is paginated. To access additional pages, append `?page={num}` to the URL where `num` is the page number.

Required Function Signature

You must implement the following function exactly as specified:

```
def bestInGenre(genre: str) -> str:
    """
    Finds the highest-rated TV series in the given genre.

    Parameters:
        genre (str): The genre to search for (e.g., 'Action', 'Comedy', 'Drama')

    Returns:
        str: The name of the highest-rated show in the genre. If there is a tie,
            returns the alphabetically lower name. Returns the name as a string.

    Notes:
        - Ties are broken by alphabetical order of the show name
        - Genre matching is case-insensitive
        - Shows can have multiple genres (comma-separated)
    """
    # Your implementation here
```

Task Challenge

Given a genre, find the series with the highest `imdb_rating`. If there is a tie, return the alphabetically lower name.

Sample

Input: Action Output: Game of Thrones

Sample Explanation

The 4 highest-rated shows in the Action genre are:

- Game of Thrones — 9.3
 - Avatar: The Last Airbender — 9.2
 - Hagane no renkinjutsushi — 9.1
 - Shingeki no kyojin — 8.9
-

3. SQL: Advertising System Failures Report

Your solution for this challenge must be in a file named `applicant_query.sql`.

As part of HackerAd's advertising system analytics, a team needs a list of customers who have a maximum number of failure events (`status = "failure"`) in their campaigns.

Requirements

- For all customers with more than 3 events with `status = 'failure'`, report the customer name and their number of failures.
- The result should be ordered by the number of failures in descending order.

Expected Output Format

The result should be in the following format:

customer	failures
Whitney Ferrero	6

Table Schemas

Table: customers

Column	Type	Description
id	SMALLINT	Customer ID
first_name	VARCHAR(64)	Customer first name
last_name	VARCHAR(64)	Customer last name

Table: campaigns

Column	Type	Description
--------	------	-------------

id	SMALLINT	Campaign ID
customer_id	SMALLINT	Customer ID
name	VARCHAR(64)	Campaign name

Table: events

Column	Type	Description
dt	VARCHAR(19)	Event timestamp
campaign_id	SMALLINT	Campaign ID
status	VARCHAR(64)	Event status

Sample Data

customers

id	first_name	last_name
1	Whitney	Ferrero
2	Dickie	Romera

campaigns

id	customer_id	name
1	1	Upton Group
2	1	Roob, Hudson and Rippin
3	1	McCullough, Rempel and Larson
4	1	Lang and Sons
5	2	Ruecker, Hand and Haley

events

dt	campaign_id	status
2021-12-02 13:52:00	1	failure
2021-12-02 08:17:48	2	failure
2021-12-02 08:18:17	2	failure
2021-12-01 11:55:32	3	failure
2021-12-01 06:53:16	4	failure
2021-12-02 04:51:09	4	failure

2021-12-01 06:34:04	5	failure
2021-12-02 03:21:18	5	failure
2021-12-01 03:18:24	5	failure
2021-12-02 15:32:37	1	success
2021-12-01 04:23:20	1	success
2021-12-02 06:53:24	1	success
2021-12-02 08:01:02	2	success
2021-12-01 15:57:19	2	success
2021-12-02 16:14:34	3	success
2021-12-02 21:56:38	3	success
2021-12-01 05:54:43	4	success
2021-12-02 17:56:45	4	success
2021-12-02 11:56:50	4	success
2021-12-02 06:08:20	5	success

4. AI-Powered Data Classification Engine

Your evaluator will provide a separate Challenge 4 document specifying the language you must use.

Build an AI-powered data classification engine that uses an OpenAI-compatible LLM to classify short text samples by sensitivity, category, or risk.

You will be asked to run your demo live against a real LLM endpoint during the live walkthrough. Bring an OpenRouter (or compatible) API key.